

Computational Practices: Sound and Image Processing

BSc Creative Computing

Your Name: Zheng Jie Wang

Unit:25029011

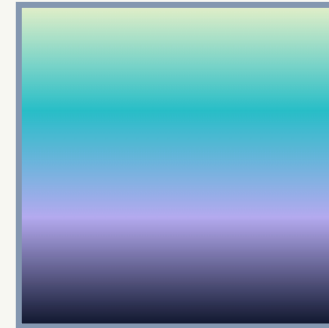
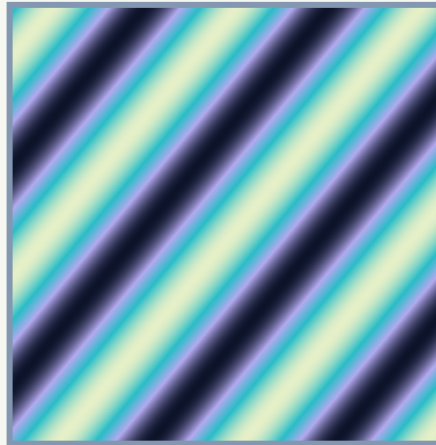
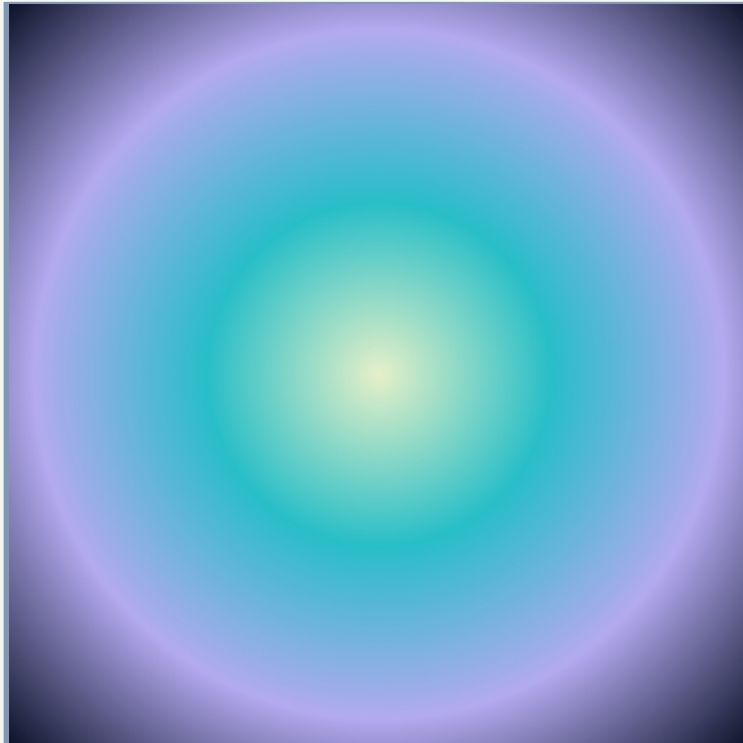
Year 1

GitHub Repository Link:

<https://github.com/2823555062-a11y/Computational-Practices-Sound-and-Image-Processing-sketch/tree/b2b953049329b87cf79467d3224e5319ccca0070/Computational%20Practices%20Sound%20and%20Image%20Processing>

Week 1

Pixels and Colour



```
pixels[x + y * width] = c;}
```

```
//Distance between the current pixel and the centre point  
float distanceFromCenter = dist(x, y, centerX, centerY);  
//Map the distance to a value between 0 and 1 to create a circular gradient  
float t = map(distanceFromCenter, 0, maxDistance, 0, 1);  
t = constrain(t, 0, 1);
```

```
float localT = map(t, 0, 0.33, 0, 1);  
c = lerpColor(softWhite, neonCyan, localT);  
}
```

```
float localT = map(t, 0.66, 1, 0, 1);  
c = lerpColor(lavenderPurple, midnightBlue, localT);  
}
```

```
float localT = map(t, 0.33, 0.66, 0, 1);  
c = lerpColor(neonCyan, lavenderPurple, localT);}
```

Techniques Used:

Techniques Used:
loadPixels() / updatePixels()
nested for loops
RGB colour values
map()
radial, wave and linear gradients

Task:

This week's task was to design a visual identity for a computational art gallery using Processing. I explored how code can be used to create colour, gradient and graphic compositions.

What I did:

I first created a basic template, and then I used this basic template to draw circles and create interlaced patterns.

I used nested loops to calculate every pixel on the canvas. `dist()` and `map()` create the radial gradient, while `lerpColor()` blends the colours smoothly and writes the final result into the pixel array.

Reflection:

This task helped me understand how changing pixel values can create different visual systems.

Week2

Manipulating and Analysing Pixels

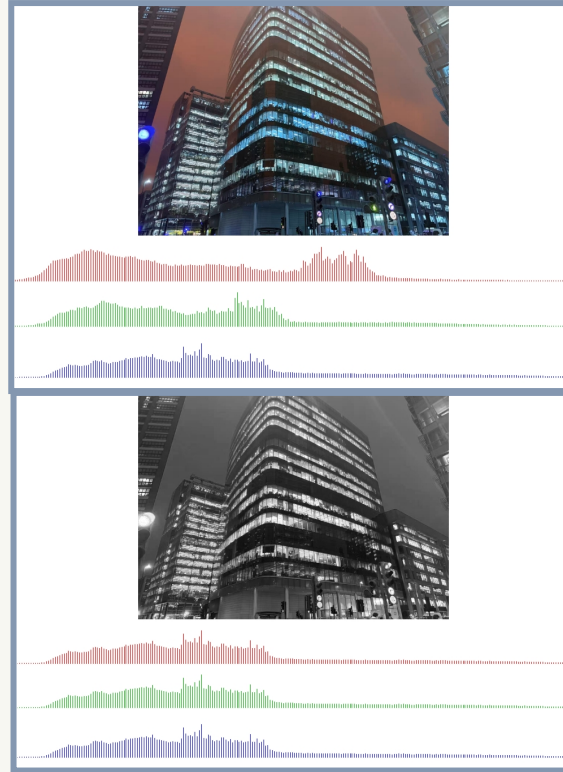


```
for (int i = 0; i < sample.pixels.length; i++) {  
    color c = sample.pixels[i];  
  
    int r = int(red(c));  
    int g = int(green(c));  
    int b = int(blue(c));  
  
    // Swap red and blue channels: RGB becomes BGR  
    sample.pixels[i] = color(b, g, r);  
}  
  
sample.updatePixels();  
}
```

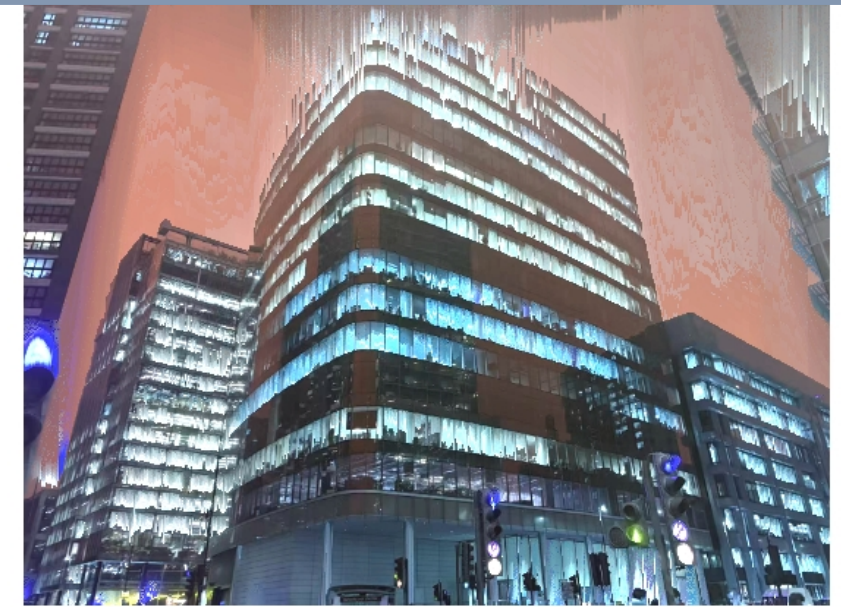
```
// Add RGB  
redHistogram[r]++;  
greenHistogram[g]++;  
blueHistogram[b]++;  
}  
  
//create three histograms  
int[] redHistogram = new int[256];  
int[] greenHistogram = new int[256];  
int[] blueHistogram = new int[256];
```

Techniques Used:

PImage, loadImage(),
loadPixels(), pixels[], red() /
green() / blue(), arrays, for loop,
RGB histogram



Coding: This code reads each pixel's RGB values and changes them directly to create a colour channel swap effect. This code counts how often each red, green and blue value appears in the image, creating an RGB histogram for colour analysis.



Colour Swapped + Brightened + Pixel Sorting Melt

What I did: My process of experimentation went from left to right. In the first image, I tried to retrieve the color values. In the second image, I inverted the colors. In the third image, I changed the color values to black and white. In the fourth image, I created the effects of melting drips and scattering particles.

Week3

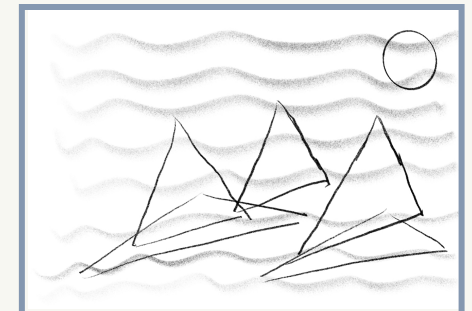
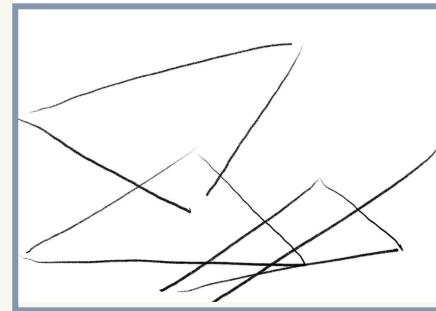
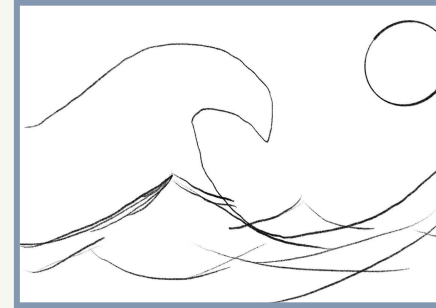
Vector Graphics



Techniques

Used:

- ellipse()
- line()
- beginShape() / vertex()
- for loops
- RGB colour values
- mouse interaction
- layered composition



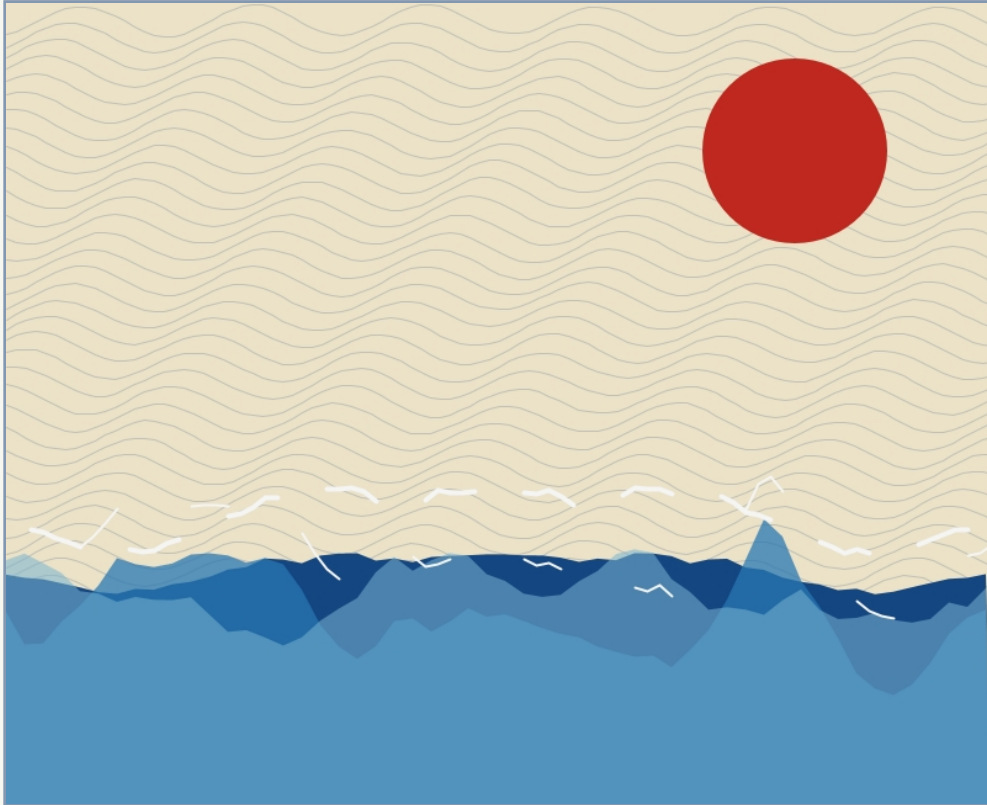
Katsushika Hokusai (c.1830–32)
Under the Wave off Kanagawa
(Kanagawa oki nami ura), also
known as The Great Wave,
from the series Thirty-six Views
of Mount Fuji. The Metropolitan
Museum of Art. Available at:
<https://www.metmuseum.org/art/collection/search/45434>
(Accessed: 2 June 2026).

What I did: The main objective of this week is to create an abstract work. The requirements mentioned serrations, which immediately made me think of waves. And waves are most famously depicted in the waves of Ukiyo-e. I immediately thought of abstracting the waves from Ukiyo-e.

I will first trace the outline of the waves, then use triangles to generalize the shape of the outline. Finally, I will re-stack and combine the generalized triangles to create the new waves I desire.

Week4

Filtering Images



Task:In this project, I created a 2D animated wave scene using Processing. I used Perlin Noise to generate natural wave movement and foam lines. The background uses repeated sine wave lines to create a decorative sea-like pattern. I also added mouse interaction, so when the mouse moves left or right, the waves move in the same direction. This helped me understand how noise can be used to create natural and organic motion.

```
//left or right controls wave direction
float mouseDirection = mouseX - pmouseX;
//Let the waves move a little slower
waveMove = waveMove + mouseDirection * 0.008;
```

Coding one:This part uses the mouse movement to control the direction of the wave animation.

```
// main shape
float waveLine = sin(x * 0.045 + time * 1.8 + y * 0.03) * 12;

// small noise
float n = noise(x * 0.01, y * 0.02, time * 0.5);
float noiseLine = map(n, 0, 1, -4, 4);
vertex(x, y + waveLine + noiseLine);}
```

Coding two:This part creates dense background wave lines using sine waves and Perlin noise.

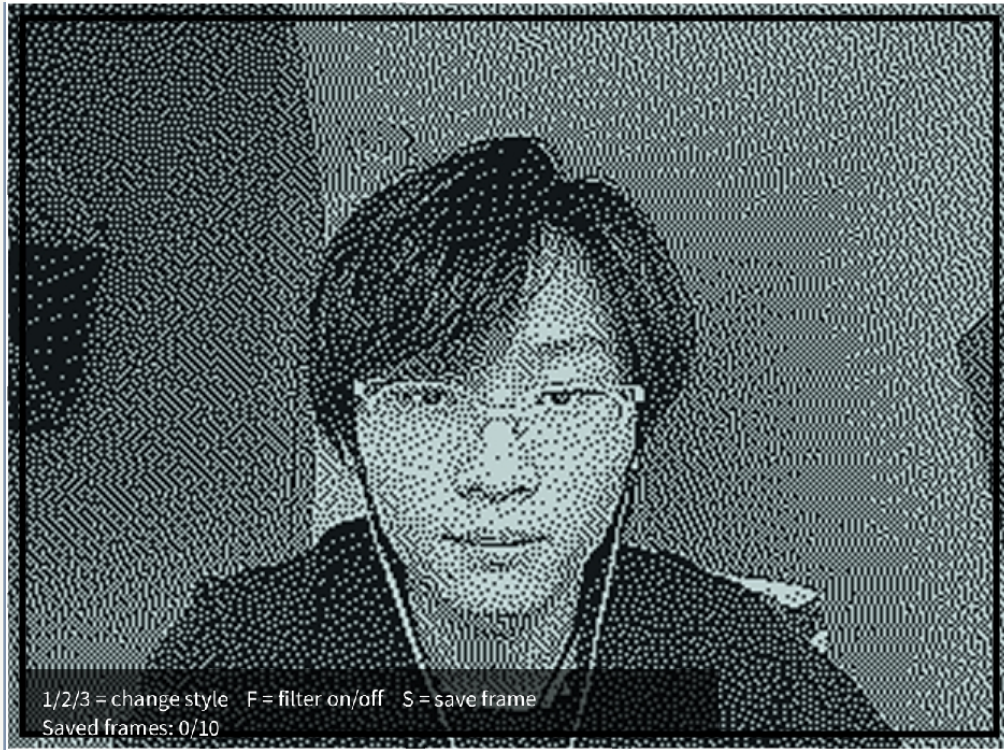
```
//Generate the top of the waves using noise
for (int x = 0; x <= width; x += 15) {
  float n = noise(x * 0.008 - waveMove, time * 0.5);
  float y = map(n, 0, 1, 400, 500);
  vertex(x, y);
}
```

Coding three:This part uses Perlin noise to generate the natural shape of the main wave.

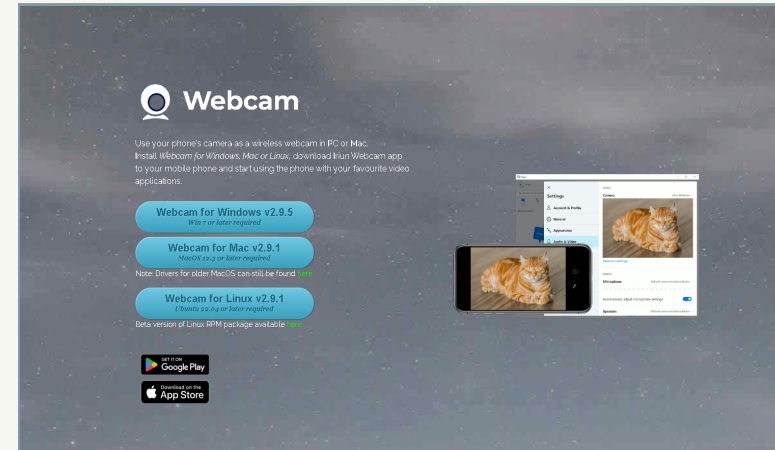
What I did:For this week's coding assignment, I referred to the previous one and continued to use wave patterns. However, I want to make it more fluid and interactive, while also incorporating the requirement to use noise.

Week6

Dithering and Convolution



I use the Processing Video library to capture real-time camera input and convert it into processed visual output. I experimented with a dithering effect, transforming the camera image into a pixel-based black-and-white style image. This is quite similar to the style of a sculpture I designed previously, representing a deeper exploration from that assignment. The advantage over that previous project is that this one can process input images in real time for output. This helps me connect real-time video with image processing.



The process of connecting the camera was very difficult; either the camera could not be detected or there was a problem with the camera component version. I also tried using an external camera with my phone, and it was successful.

Camera 3D | Alter P3D Rendering to produce Stereosc...
GL Video | Hardware accelerated video on the Raspbe...
Image processing algorithms | Implementations of basi...
Luna Video Mapping | A video mapping library with a u...
Syphon | This library allows the sharing of video and i...
Video Export | Simple video file exporter.
Video Library for Processing 3 | GStreamer-based vide...
Video Library for Processing 4 | GStreamer-based vide...
VLCJVideo | VLCJ binding for Processing.

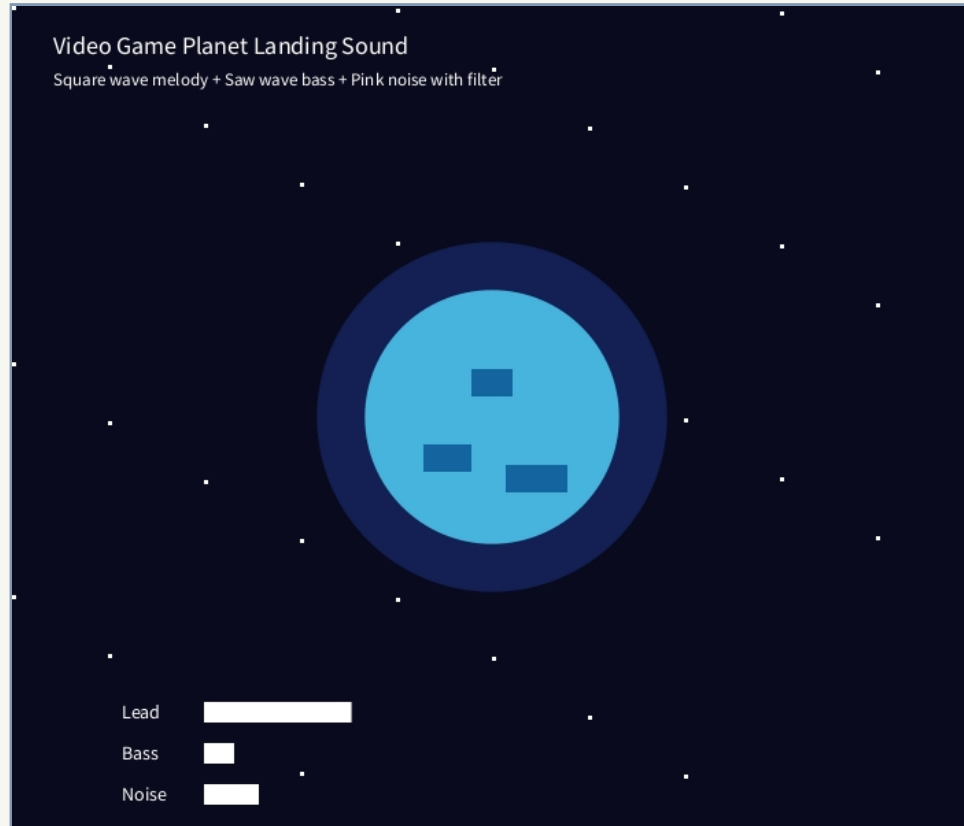
Jim Schmitz
Gottfried Haider
Nick 'Milchreis' Müller
Daniel Corbani
Andres Colubri, Tom Butterwort...
Abe Pazos
 **The Processing Foundation**
 **The Processing Foundation**
Caldas Lopes

Techniques Used:
Processing Video library
Capture
cam.read()

loadPixels()
brightness values
dithering
real-time image processing
camera input

Week7

Digital Sound and Oscillation



What I did:

I created an interactive sound sketch using oscillators from the Processing Sound library. I connected sound parameters such as frequency and amplitude with visual changes on the screen, so the image and sound respond together.

```
//Create the sounds
lead = new SqrOsc(this);
bass = new SawOsc(this);
noise = new PinkNoise(this);
filter = new LowPass(this);

//Put the noise through the filter
filter.process(noise);

//Start sounds
lead.play();
bass.play();
noise.play();
```



9iOldGame (n.d.) FC/MSX2
Contra. Available at:
<https://9ioldgame.com/tag/Contra/> (Accessed: 2 June 2026).

Coding one: This code creates the main synthetic sound sources: a square wave melody, saw wave bass, pink noise and a low-pass filter.

```
//Frequency Modulation
void makeSound() {
  float move = sin(frameCount * 0.15) * 8;
  lead.freq(leadFreq + move);

  //Play melody every 12 frames
  if (frameCount % 12 == 0) {
    leadFreq = melody[melodyNumber];
    leadVolume = 0.2;

    //Next
    melodyNumber = melodyNumber + 1;

    //If the melody finishes playing, start from the beginning
    if (melodyNumber >= melody.length) {melodyNumber = 0;}

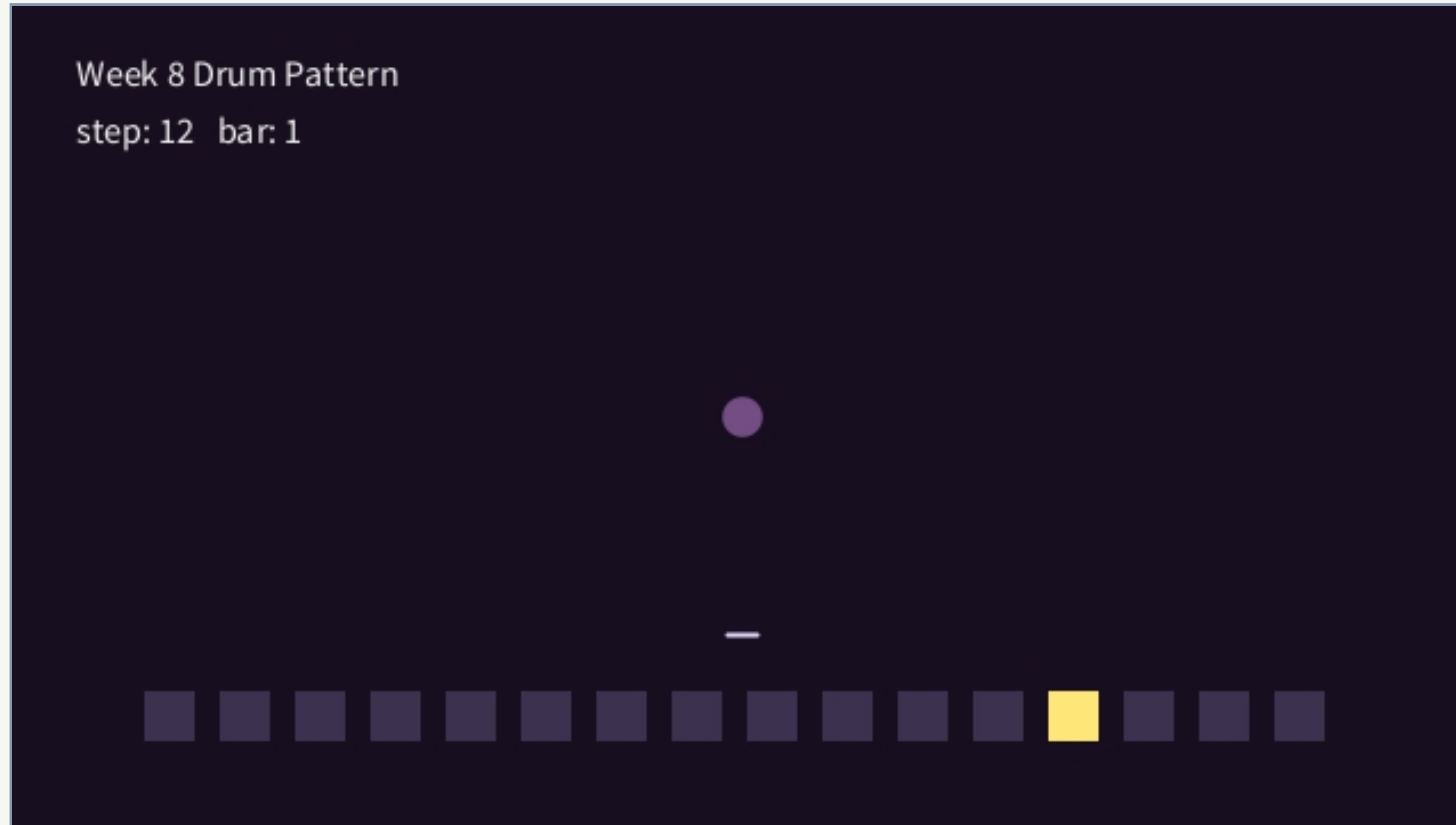
    //Make the planet a bit bigger
    planetSize = 190;}
}
```

Coding two: This code changes the melody over time and makes the planet expand when a new note is played.

Techniques Used
Processing Sound library
SinOsc / oscillator sound synthesis
frequency and amplitude control
map()
mouse interaction
animated visual feedback
basic sound-image relationship

Week8

Algorithmic music and sampling



What I did:

I created an algorithmic drum pattern in Processing using four sound samples: kick, snare, hihat and clap. Instead of playing the sounds randomly, I used a step-based rhythm system so each sample plays at a specific moment in the loop. The kick creates the main beat, the snare gives the rhythm structure, and the hihat and clap add faster details. I also adjusted the pattern to create a darker and more atmospheric sound, inspired by an electronic music intro rather than a simple drum exercise.

```
if (step == 0 || step == 7) {
  kick.play();
  kickSize = 150;
}

if (bar > 1 && step == 10) {
  kick.play();
  kickSize = 110;
}

if (step == 4 || step == 12) {
  snare.play();
  snareSize = 120;
}

if (step % 2 == 0) {
  if (random(1) < 0.8) {
    hihat.play();
    hatSize = 130;
  }
}

if (bar > 1 && step == 3) {
  if (random(1) < 0.5) {
    hihat.play();
    hatSize = 80;
  }
}

if (bar % 2 == 1 && step == 12) {
  clap.play();
  flash = 130;
}
```

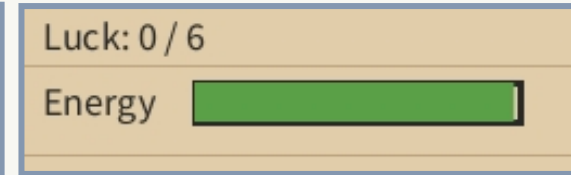
This code triggers different drum samples at specific steps. Kick, snare, hihat and clap are arranged into a looped rhythm instead of playing randomly.

Techniques Used:

- Processing Sound library
- SoundFile
- sound samples
- modulo %
- step counter
- arrays / rhythm pattern
- frameCount or timing control

Week9

Vectors and movement



Luck counter: shows collected clovers
Energy bar: shows remaining energy



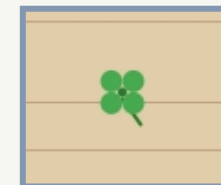
Controls: keyboard input system

Task:

This week's task was to create a simple interactive game in Processing. The game needed to include character movement, interaction, rules and visual feedback.

What I did:

I made a game where the player controls a worm to collect 6 lucky clovers. The worm can bend left or right and move forward using keyboard input. The player needs to manage energy while avoiding failure. When all clovers are collected, the game changes to a win screen with falling petals.



Clover: collectible object

Week9

Vectors and movement

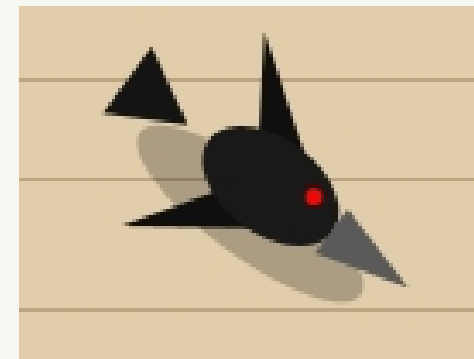


Bird Enemy Evolution

The bird was designed as the main enemy in the game. Its job is to create pressure while the player controls the worm and tries to collect the clovers. I wanted the bird to feel dangerous, but I also kept the shape.

At first, the bird was only a basic flying shape. It did not feel threatening enough, so I changed the design step by step. I kept the body as one simple shape instead of splitting it into different body parts. Then I added a sharper beak pointing forward, because the direction of the beak makes the bird look like it is attacking the worm. I also kept the tail and added a pair of wings to make the bird more recognisable as a flying enemy.

In the final version, the red eye, sharp beak, dark body, wings and tail make the bird feel more aggressive. It becomes a visual warning for the player and helps make the game more tense.



Video

https://artslondon-my.sharepoint.com/:f:/g/personal/z_wang0220256_arts_ac_uk/IgB8r3iFLuWbSJi4smteFjl7ARZBKcnEeTVF7trycXLMEc4?e=CbJjn2

GitHub Repository Link:

<https://github.com/2823555062-a11y/Computational-Practices-Sound-and-Image-Processing-sketch/tree/b2b953049329b87cf79467d3224e5319ccca0070/Computational%20Practices%20Sound%20and%20Image%20Processing>

Thank You